

Data: 20/09/2026

1. Introdução

Este documento descreve o projeto entregue para comprovar o aprendizado da Fase 1 do curso Postech Arquitetura e Desenvolvimento em Java.

Utilizei a ferramenta Apicurito para projetar o contrato da aplicação antes de desenvolver. O arquivo gerado através dessa ferramenta está disponível no GitHub em formato “YAML” com o nome “TodosRestaurantes.yaml”.

2. Definição do Problema

Um grupo de restaurantes, visando otimizar o custo com aplicações para gerenciamento do negócio e atendimento dos clientes, contratou o desenvolvimento de um sistema único e compartilhado.

Nesse sistema, os clientes escolherão os restaurantes com base na comida fornecida.

A versão completa do sistema deve atender dois perfis:

- ⇒ Dono de restaurante: funções para gestão eficiente das operações do restaurante;
- ⇒ Cliente do restaurante: funções para pesquisar a comida disponível, realizar pedidos e avaliar o atendimento.

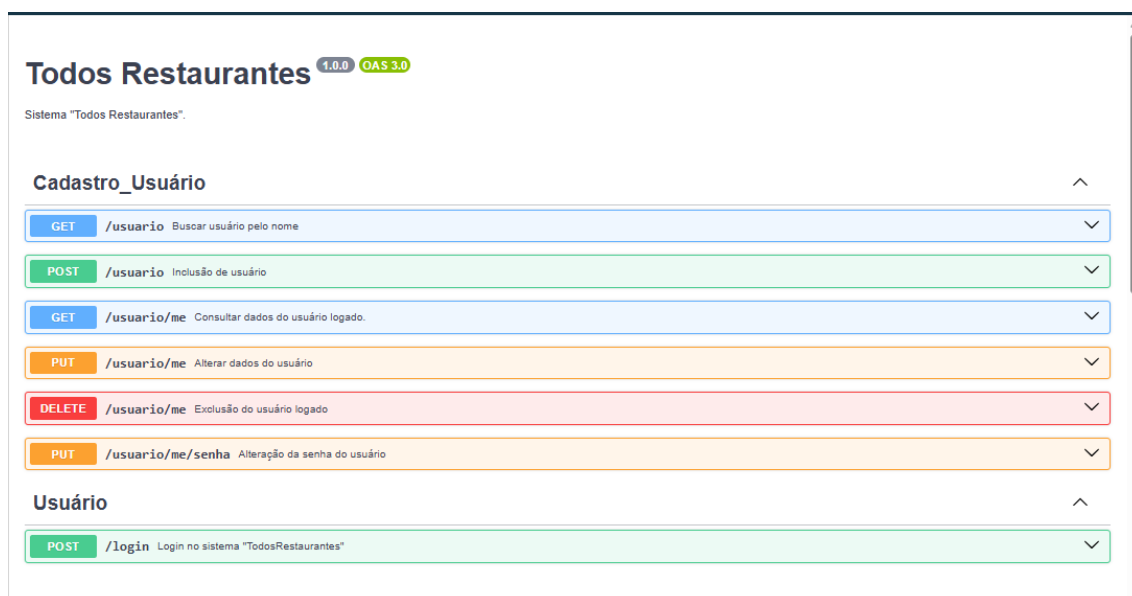
O desenvolvimento foi dividido em fases e a primeira fase deve entregar um *backend* com os seguintes requisitos funcionais:

- ⇒ Manutenção de um Cadastro de Usuário, contemplando os seguintes atributos: nome, e-mail, senha, data da última alteração e endereço, sendo que a unicidade do e-mail deve ser garantida;
- ⇒ Para acessar as funções do sistema, o usuário deve efetuar LOGIN informando o e-mail e senha;
- ⇒ A troca de senha deve ser permitida, porém, em *endpoint* distinto do que efetua a manutenção dos demais dados cadastrais.

Os seguintes requisitos não funcionais devem ser atendidos:

- ⇒ A aplicação deve ser dockerizada, utilizando Docker Compose para orquestração com um banco de dados relacional;
- ⇒ Utilizar as boas práticas do Spring Boot, SOLID e Orientação a Objetos;
- ⇒ Os endpoints devem ser documentados com Swagger/Open API.

3. Projeto dos endpoints a serem desenvolvidos



A estratégia de versionamento será utilizada, mas não será explícita na URI. A versão que está sendo consumida deverá ser informada através do header "x-version".

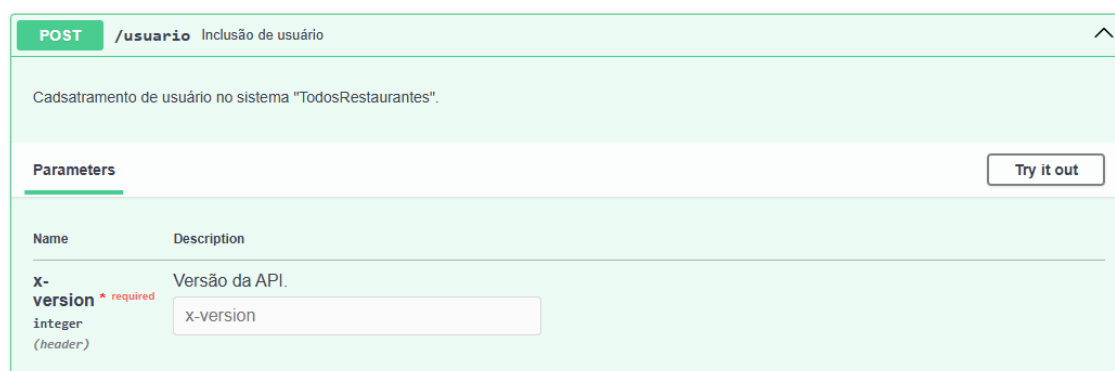
⇒ Inclusão de Usuário:

A inclusão do usuário poderá ser efetuada através da ação "Criar conta" de um aplicativo FrontEnd. Para essa função, não será exigido LOGIN.

O perfil do usuário será identificado pelo atributo "tipoUsuario", que será um ENUM com os valores "CLIENTE_RESTAURANTE" e "DONO_RESTAURANTE". Neste primeiro momento, tanto cliente como dono de restaurante possuem o mesmo conjunto de dados e podem se cadastrar somente com um dos perfis.

Se o e-mail informado já existir na base de Usuários, a requisição não será atendida e será respondido status 400 (Bad Request), com a mensagem "Não é possível cadastrar o usuário" para não deixar claro se o e-mail utilizado existe ou não.

Endpoint para Inclusão de usuário:



⇒ Para inclusão de cliente:

Request body required application/json

Requisição para inclusão, alteração e exclusão de usuário no Cadastro de Usuários do sistema "TodosRestaurantes".

Examples:

Inclusão de CLIENTE

Example Value | Schema

```
{
  "nome": "some text",
  "endereco": {
    "tipoLogradouro": "RUA",
    "nomeLogradouro": "some text",
    "numero": "some text",
    "bairro": "some text",
    "cepCompleto": "some text",
    "cidade": "some text",
    "uf": "SP"
  },
  "tipoUsuario": "CLIENTE_RESTAURANTE",
  "login": {
    "senha": "teste123",
    "email": "endereco@email.com"
  }
}
```

⇒ Para inclusão de dono de restaurante:

Request body required application/json

Requisição para inclusão, alteração e exclusão de usuário no Cadastro de Usuários do sistema "TodosRestaurantes".

Examples:

Inclusão de DONO DE RESTAURANTE

Example Value | Schema

```
{
  "nome": "some text",
  "endereco": {
    "tipoLogradouro": "TRAVESSA",
    "nomeLogradouro": "some text",
    "numero": "some text",
    "bairro": "some text",
    "cepCompleto": "some text",
    "cidade": "some text",
    "uf": "SP"
  },
  "tipoUsuario": "DONO_RESTAURANTE",
  "login": {
    "senha": "senha123",
    "email": "endereco@email.com.br"
  }
}
```

Se a solicitação for atendida com sucesso, será respondido http 201 (Created), pois não é necessário retornar dados.

Se ocorrer um erro de negócio na inclusão do usuário, por exemplo, tentativa de incluir um usuário com e-mail já cadastrado, será retornado http 400 (Bad Request) com a mensagem "Não é possível cadastrar o usuário". Nesse caso, a mensagem não deixa claro o motivo para evitar validação implícita do endereço de e-mail.

Se houver erro no acesso ao banco de dados, será retornado http 500.

Code	Description	Links
201	Usuário incluído com sucesso.	No links
400	Erro de negócio.	No links
	Media type: application/json Examples: Não é possível incluir o usuário Example Value Schema <pre>{ "mensagem": "Não é possível incluir o usuário", "status": 400 }</pre>	
500	Erro técnico no sistema.	No links
	Media type: application/json Examples: Sistema Indisponível Example Value Schema <pre>{ "mensagem": "Sistema Indisponível", "status": 500 }</pre>	

⇒ LOGIN no sistema:

As demais funcionalidades do sistema poderão ser acessadas somente após a autenticação do usuário através do LOGIN. Neste passo, o usuário deve informar o e-mail cadastrado no momento da ação “Criar conta” e a senha que atribuiu.

O sistema validará o e-mail e senha através de consulta ao Banco de Dados e, caso encontre correspondência exata, retornará um token JWT que terá em sua composição o e-mail e ID que identifica o usuário no sistema.

Os demais endpoints exigem o token JWT (Bearer Token) no Header, para garantir a segurança.

Se as informações do LOGIN não estiverem corretas, será retornado http 401 (Unauthorized) com a mensagem “Login incorreto”, para não deixar claro o que não está correto.

Na resposta, também será informado

Endpoint para efetuar LOGIN:

Usuário
^

POST /login Login no sistema "TodosRestaurantes"
 ^

Requisição para LOGIN no sistema "TodosRestaurantes".

Parameters
Try it out

Name	Description
x-version * required number (header)	Versão da API. x-version

Request body required application/json

Requisição para LOGIN no sistema "TodosRestaurantes".

Examples:
Solicitação de LOGIN

Example Value | Schema

```
{
  "login": {
    "email": "endereco@email.com.br",
    "senha": "teste1234"
  }
}
```

401 Dados do LOGIN não estão corretos. No links

Media type application/json Examples Dados do Login incorretos

Example Value | Schema

```
{
  "mensagem": "Login incorreto",
  "status": 401
}
```

500 Erro de sistema. No links

Media type application/json Examples Sistema Indisponível

Example Value | Schema

```
{
  "mensagem": "Sistema Indisponível",
  "status": 500
}
```

⇒ Consulta dados do usuário logado:

Consulta dados do usuário logado. Neste caso, não é necessário informar a identificação do usuário porque o sistema obtém a informação no token JWT e a consulta será efetuada diretamente pelo ID do usuário. O sistema efetuará a conferência do e-mail e ID obtidos no token.

Endpoint da Consulta de dados do usuário logado:

GET /usuario/me Consultar dados do usuário logado.

Consultar dados do usuário logado.

Parameters Try it out

Name	Description
x-version * required integer (header)	Versão da API. x-version

Responses

Code	Description	Links
200	Consulta de dados do usuário logado efetuada com sucesso.	No links

Media type: Examples:

Controls Accept header.

Example Value | Schema

```

{
  "nome": "some text",
  "endereco": {
    "tipoLogradouro": "TRAVESSA",
    "nomeLogradouro": "some text",
    "numero": "some text",
    "bairro": "some text",
    "cepCompleto": "some text",
    "cidade": "some text",
    "uf": "GO"
  },
  "ultimaManutencao": "2018-02-10T09:30Z",
  "email": "some text"
}

```

Responses

Code	Description	Links
200	Consulta de dados do usuário logado efetuada com sucesso.	No links

Media type: Examples:

Controls Accept header.

Example Value | Schema

```

{
  "nome": "some text",
  "endereco": {
    "tipoLogradouro": "RUA",
    "nomeLogradouro": "some text",
    "numero": "some text",
    "bairro": "some text",
    "cepCompleto": "some text",
    "cidade": "some text",
    "uf": "PE"
  },
  "ultimaManutencao": "2018-02-10T09:30Z",
  "email": "some text"
}

```

400	Erro de negócio.	No links
Media type: application/json		
Example Value Schema		
<pre> { "mensagem": "string", "status": 0 } </pre>		
500	Erro técnico no sistema.	No links
Media type: application/json		
Example Value Schema		
<pre> { "mensagem": "string", "status": 0 } </pre>		

⇒ Altera dados do usuário logado:

Altera os dados do usuário logado. Neste caso, não é necessário informar a identificação do usuário porque o sistema obtém a informação no token JWT e a consulta será efetuada diretamente pelo ID do usuário.

Será permitido alterar o e-mail. Porém, após alteração, o usuário terá que efetuar o LOGIN novamente.

Endpoint para alteração dos dados do usuário:

PUT **/usuario/me** Alterar dados do usuário

Alteração de dados do usuário logado no sistema "TodosRestaurantes".

Parameters Try it out

Name	Description
x-version * required integer (header)	Versão da API. x-version

Request body required application/json

Requisição de alteração dos dados do usuário.

Examples:
Alterar dados do Cliente

Example Value | Schema

```
{
  "nome": "some text",
  "endereco": {
    "tipoLogradouro": "TRAVESSA",
    "nomeLogradouro": "some text",
    "numero": "some text",
    "bairro": "some text",
    "cepCompleto": "some text",
    "cidade": "some text",
    "uf": "GO"
  },
  "email": "some text"
}
```

Request body required application/json

Requisição de alteração dos dados do usuário.

Examples:
Alterar dados do Dono do Restaurante

Example Value | Schema

```
{
  "nome": "some text",
  "endereco": {
    "tipoLogradouro": "AVENIDA",
    "nomeLogradouro": "some text",
    "numero": "some text",
    "bairro": "some text",
    "cepCompleto": "some text",
    "cidade": "some text",
    "uf": "ES"
  },
  "email": "some text"
}
```

400

Erro de negócio

No links

Media type

application/json

Example Value | Schema

```
{
  "mensagem": "string",
  "status": 0
}
```

500

Erro técnico.

No links

Media type

application/json

Example Value | Schema

```
{
  "mensagem": "string",
  "status": 0
}
```

⇒ **Exclui usuário logado:**

Exclui usuário logado. Neste caso, não é necessário informar a identificação do usuário porque o sistema obtém a informação no token JWT.

Endpoint para exclusão do usuário:

DELETE

/usuario/me

Exclusão do usuário logado

Solicia exclusão do usuário logado.

Parameters

Try it out

Name	Description
x-version * required integer (header)	Versão da API. x-version

Responses		
Code	Description	Links
204	Exclusão do usuário efetuada com sucesso.	No links
400	Erro de negócio. Media type application/json Example Value Schema <pre>{ "mensagem": "string", "status": 0 }</pre>	No links
500	Erro técnico. Media type application/json Example Value Schema <pre>{ "mensagem": "string", "status": 0 }</pre>	No links

⇒ Alteração de senha:

A alteração de senha só é possível com o usuário logado. Neste caso, não é necessário informar a identificação do usuário porque o sistema obtém a informação no token JWT. O Login atual é solicitado para confirmação dos dados.

Após a alteração da senha, o usuário deverá efetuar o LOGIN novamente, para uma nova validação.

Endpoint para alteração da senha:

PUT /usuario/me/senha Alteração da senha do usuário

Alteração da senha do usuário logado no sistema "TodosRestaurante".

Parameters
Try it out

Name	Description
x-version * required integer (header)	Versão da API. x-version

Request body required

application/json

Requisição para alteração da senha do usuário logado no sistema.

Examples:

Troca de senha

Example Value | Schema

```
{
  "loginAtual": {
    "senha": "teste123",
    "email": "endereco@email.com"
  },
  "novaSenha": "teste124"
}
```

Responses		
Code	Description	Links
204	Troca de senha efetuada com sucesso.	No links
400	Erro de negócio.	No links
	Media type application/json Example Value Schema <pre>{ "mensagem": "string", "status": 0 }</pre>	
500	Erro técnico.	No links
	Media type application/json Example Value Schema <pre>{ "mensagem": "string", "status": 0 }</pre>	

⇒ Busca de usuário por nome:

A busca de usuário por nome só pode ser efetuada por um usuário logado. Neste momento, somente usuário do tipo “DONO_RESTAURANTE” pode efetuar a busca e abrangendo somente usuários do tipo “CLIENTE_RESTAURANTE”. O sistema obterá o ID do usuário no token JWT para validar se é do tipo esperado.

Cadastro_Usuário

GET **/usuario** Buscar usuário pelo nome

Buscar usuários por nome. Função disponível somente para usuários logados com o perfil "DONO RESTAURANTE". Busca apenas usuários do tipo "CLIENTE".

Parameters Try it out

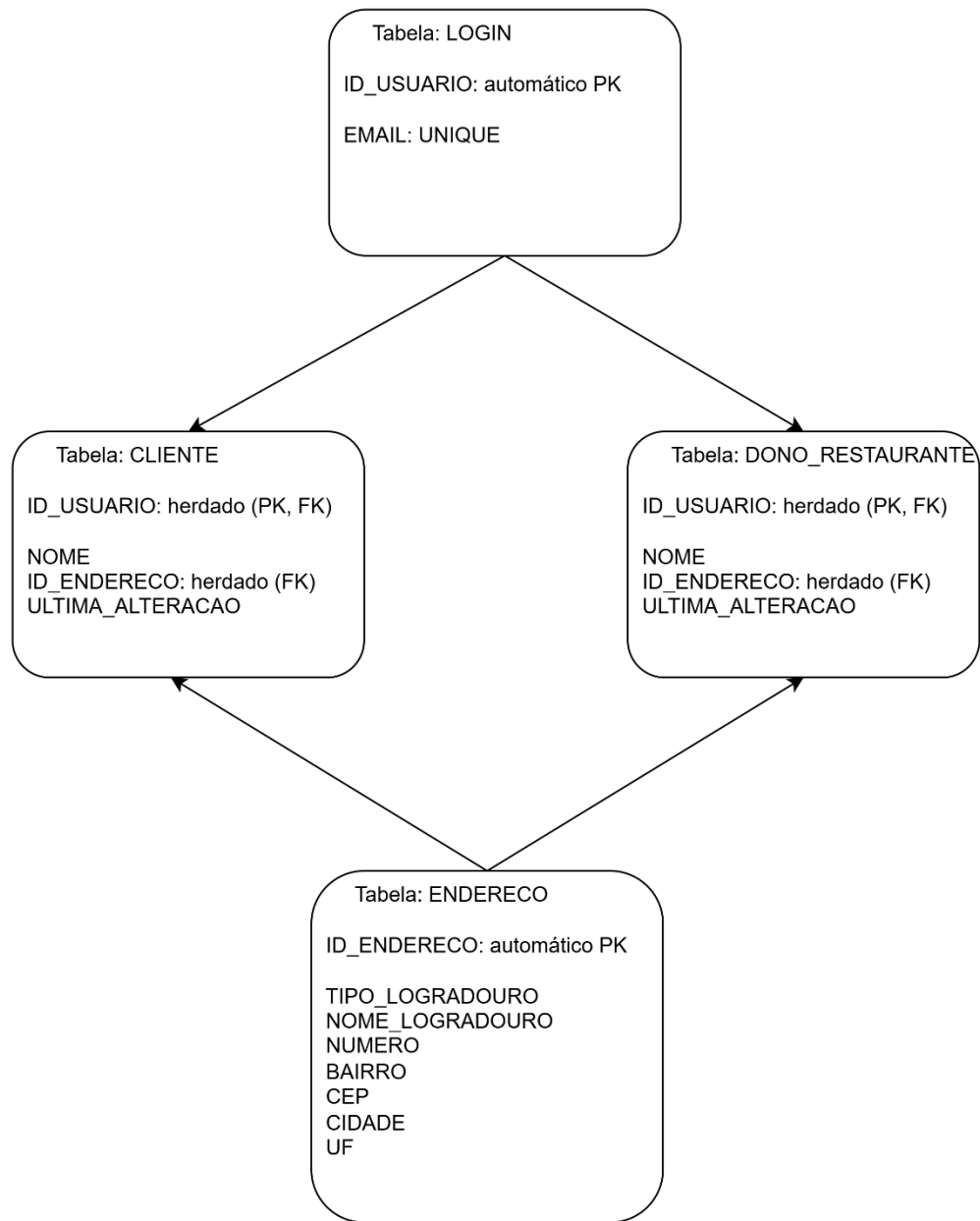
Name	Description
nome * required string (query)	Argumento para busca de usuários por nome. nome
x-version * required integer (header)	Versão da API. x-version

Responses

Code	Description	Links
200	Lista de usuários do perfil "CLIENTE" que atendem ao argumento de busca. Media type: application/json Examples: Lista de usuários Controls Accept header: Example Value Schema <pre>{ "usuarios": [{ "nome": "some text", "endereco": { "tipologradouro": "ALAMEDA", "nomelogradouro": "some text", "numero": "some text", "bairro": "some text", "cepCompleto": "some text", "cidade": "some text", "uf": "MA" }, "ultimaManutencao": "2018-02-10T09:30Z", "tipoUsuario": "CLIENTE_RESTAURANTE", "email": "some text" }, { "nome": "some text", "endereco": { "tipologradouro": "ALAMEDA", "nomelogradouro": "some text", "numero": "some text", "bairro": "some text", "cepCompleto": "some text", "cidade": "some text", "uf": "MT" }, "ultimaManutencao": "2018-02-10T09:30Z" }] }</pre>	No links
400	Fim da página	No links

400	Erro de negócio. No links Media type application/json Examples Não há usuário cadastrado com o nome solicitado Example Value Schema <pre>{ "mensagem": "Não encontrado", "status": 400 }</pre>
500	Erro técnico. No links Media type application/json Example Value Schema <pre>{ "mensagem": "string", "status": 0 }</pre>

4. Esboço da Modelagem de Dados (Modelo de dados Relacional)

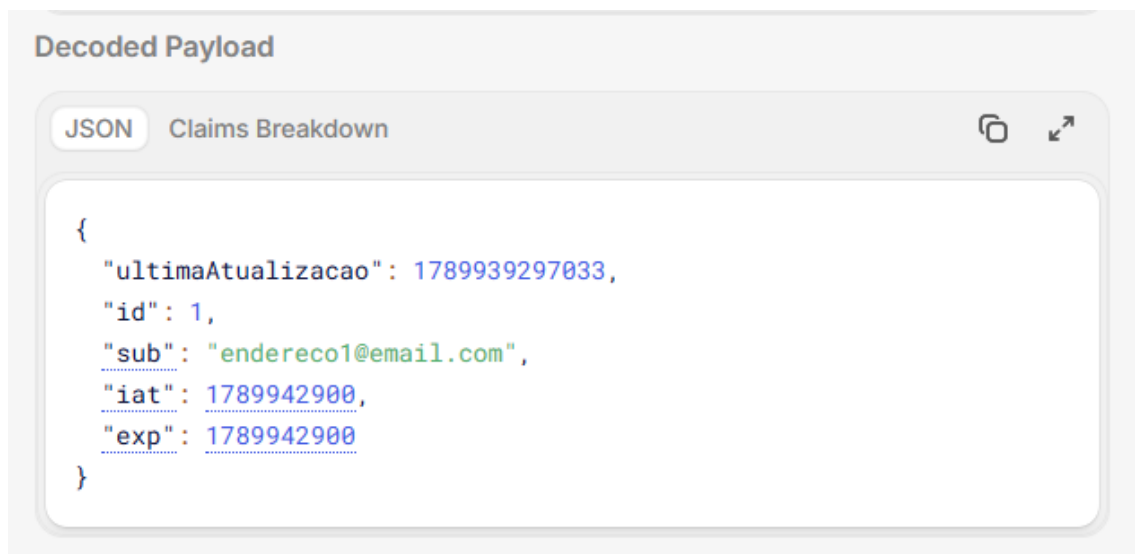


5. Informações sobre a implementação

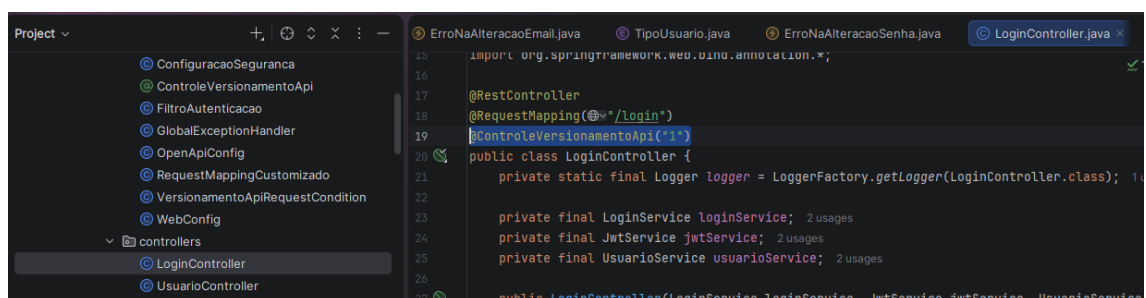
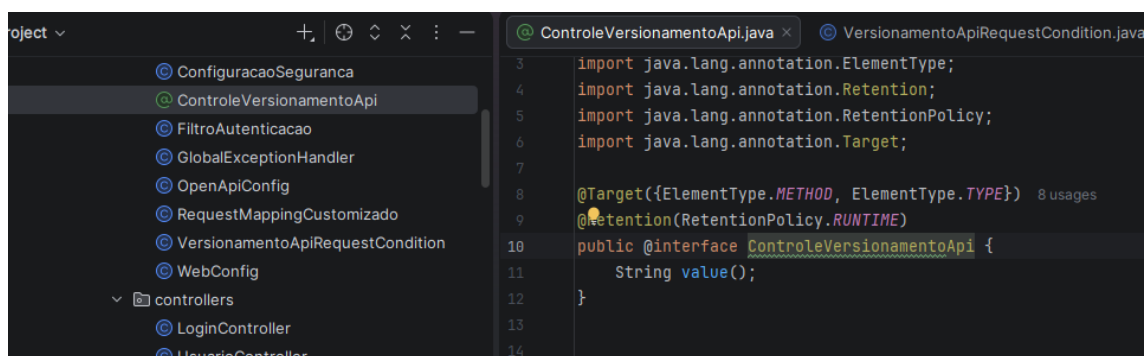
Quando ocorre troca da senha ou troca do e-mail do LOGIN, a aplicação exige que o usuário efetue uma nova autenticação, invalidando o Token atual.

Para invalidar o Token atual, o timestamp da última atualização da senha/e-mail é registrado na base LOGIN e conferido na validação do Token.

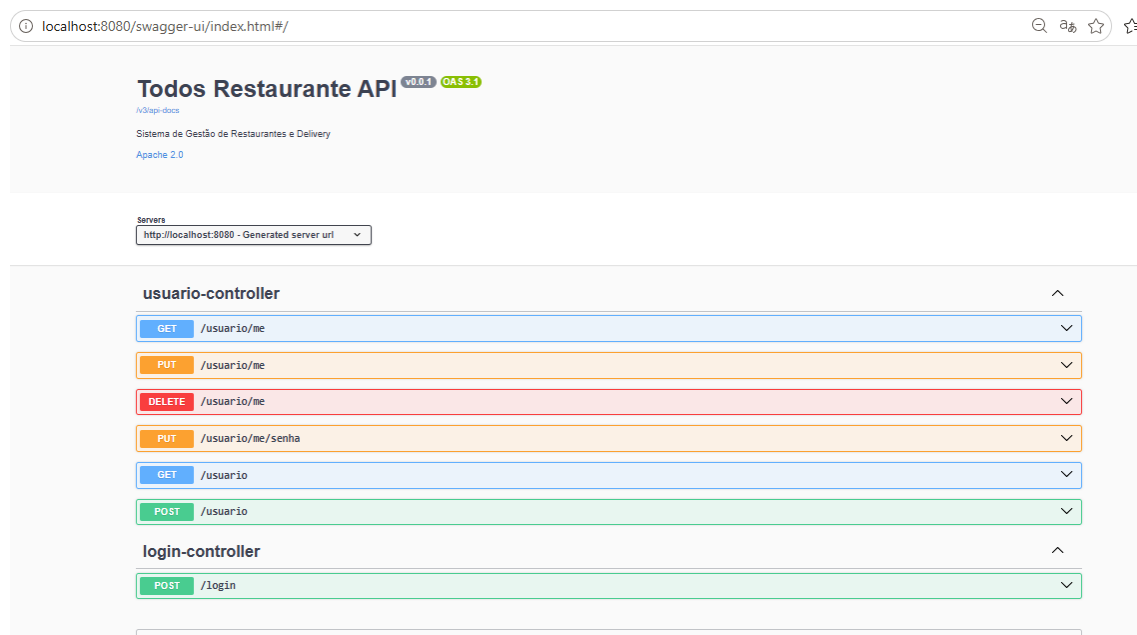
Exemplo do Token gerado:



Foi criada uma annotation para facilitar o controle das versões, uma vez que optei por não utilizar o versionamento explícito na URL.



O Swagger da aplicação pode ser visualizado através da URI “/swagger-ui/index.html”:



Para facilitar a implementação e os testes, utilizamos o banco de dados H2 em memória. Comandos utilizados para criação das tabelas:

```
CREATE TABLE LOGIN
(
    ID_USUARIO    BIGINT AUTO_INCREMENT PRIMARY KEY,
    EMAIL         VARCHAR(255) UNIQUE NOT NULL,
    SENHA         VARCHAR(100)          NOT NULL,
    ULTIMA_TROCA  TIMESTAMP(3)         NOT NULL
);

CREATE TABLE ENDERECO
(
    ID_ENDERECO BIGINT AUTO_INCREMENT PRIMARY KEY,
    TIPO_LOGRADOURO CHAR(20) NOT NULL,
    NOME_LOGRADOURO VARCHAR(60) NOT NULL,
    NUMERO CHAR(20) NOT NULL,
    BAIRRO VARCHAR(40),
    CEP CHAR(09),
    CIDADE CHAR(40),
    UF CHAR(2) NOT NULL
);

CREATE TABLE CLIENTE (
    ID_USUARIO BIGINT NOT NULL PRIMARY KEY,
    NOME VARCHAR(60) NOT NULL,
    ID_ENDERECO BIGINT NOT NULL,
    ULTIMA_ALTERACAO TIMESTAMP(3) NOT NULL,
    FOREIGN KEY (ID_USUARIO) REFERENCES LOGIN(ID_USUARIO) ON DELETE
    CASCADE,
```

```
        FOREIGN KEY (ID_ENDERECO) REFERENCES ENDERECO(ID_ENDERECO) ON
DELETE CASCADE

);

CREATE TABLE DONORESTAURANTE (
    ID_USUARIO BIGINT NOT NULL PRIMARY KEY ,
    NOME VARCHAR(60) NOT NULL,
    ID_ENDERECO BIGINT NOT NULL,
    ULTIMA_ALTERACAO TIMESTAMP(3) NOT NULL,
    FOREIGN KEY (ID_USUARIO) REFERENCES LOGIN(ID_USUARIO) ON DELETE
CASCADE,
    FOREIGN KEY (ID_ENDERECO) REFERENCES ENDERECO(ID_ENDERECO) ON
DELETE CASCADE
);
```

6. Testes

⇒ Incluindo usuário – CLIENTE:

The screenshot shows the Postman interface with a collection named 'TodosRestaurantes'. The selected request is 'POST Incluir usuário - Cliente' with the URL 'http://localhost:8080/usuario'. The request body is a JSON object representing a client user.

```
1 {
2   "nome": "Cliente do Restaurante",
3   "endereco": {
4     "tipoLogradouro": "RUA",
5     "nomeLogradouro": "Rua do Cliente",
6     "numero": "123",
7     "bairro": "Bairro do Cliente",
8     "cepCompleto": "01234-123",
9     "cidade": "Cidade do Cliente",
10    "uf": "SP"
11  },
12  "tipoUsuario": "CLIENTE_RESTAURANTE",
13  "login": {
14    "senha": "teste123",
15    "email": "endereco1@email.com"
16  }
17 }
```

The response status is 201 Created, and the response body is empty.

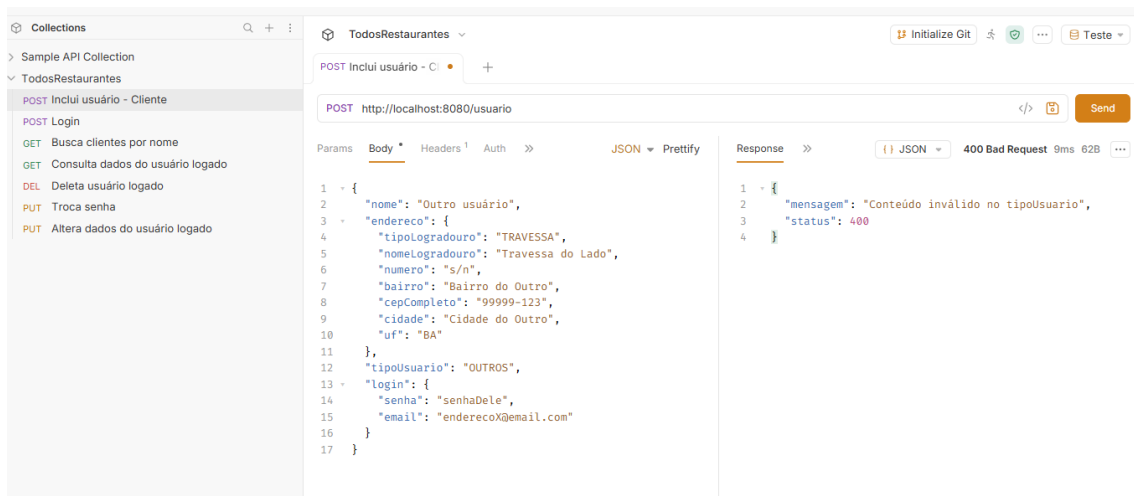
⇒ Incluindo usuário – DONO DE RESTAURANTE:

The screenshot shows the Postman interface with a collection named 'TodosRestaurantes'. The selected request is 'POST Incluir usuário - C' with the URL 'http://localhost:8080/usuario'. The request body is a JSON object representing a restaurant owner user.

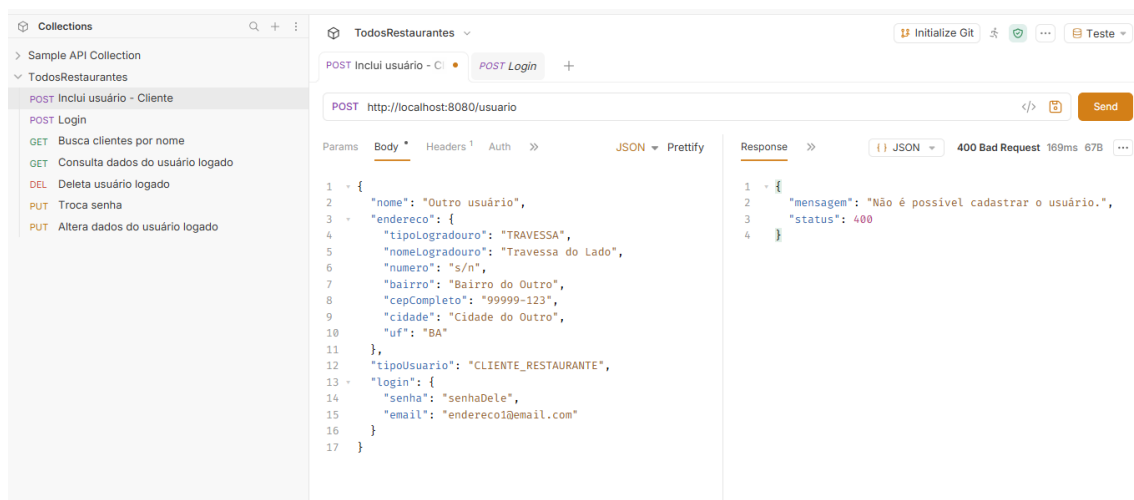
```
1 {
2   "nome": "Dono de Restaurante",
3   "endereco": {
4     "tipoLogradouro": "RUA",
5     "nomeLogradouro": "Rua do Dono do Restaurante",
6     "numero": "44",
7     "bairro": "Bairro do Dono",
8     "cepCompleto": "04444-123",
9     "cidade": "Cidade do Dono",
10    "uf": "PR"
11  },
12  "tipoUsuario": "DONO_RESTAURANTE",
13  "login": {
14    "senha": "senhaDono",
15    "email": "endereco2@email.com"
16  }
17 }
```

The response status is 201 Created, and the response body is empty.

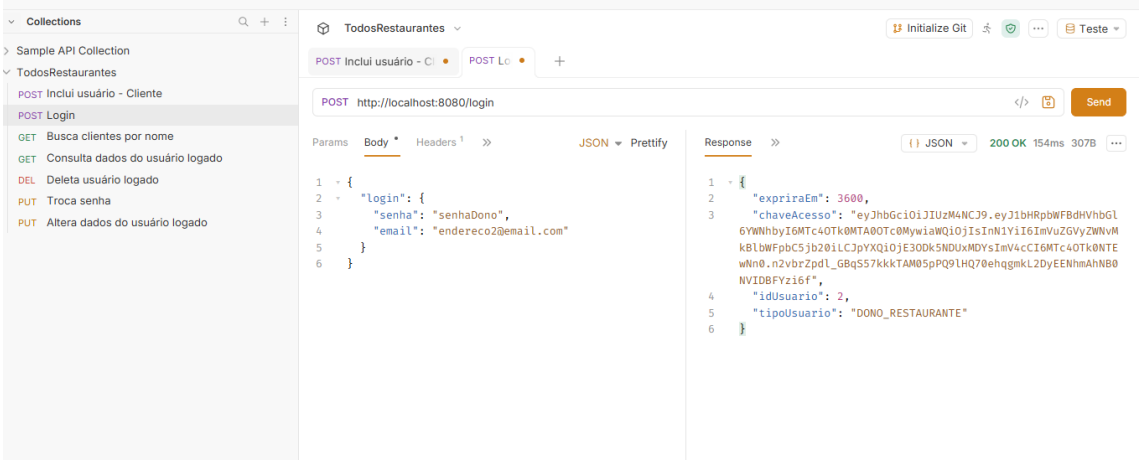
⇒ Incluindo usuário – OUTROS:



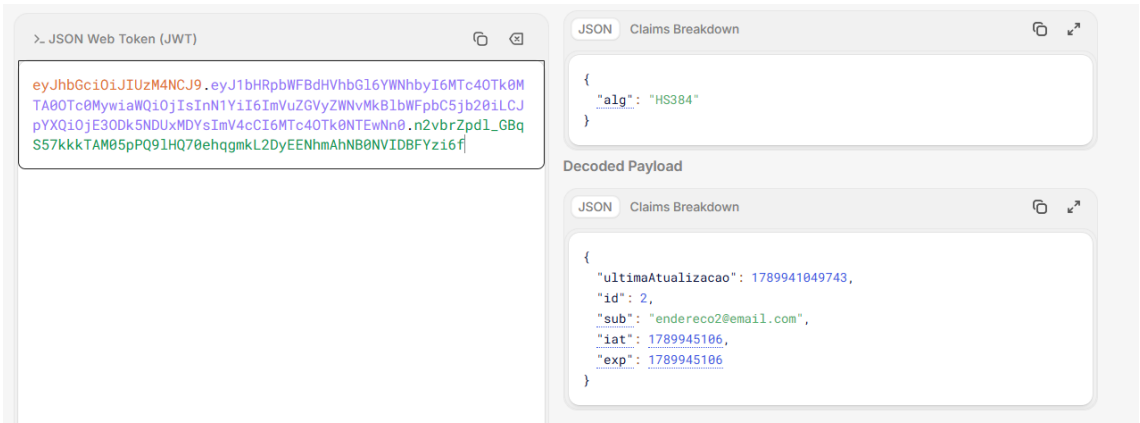
⇒ Incluindo outro usuário com e-mail já cadastrado:



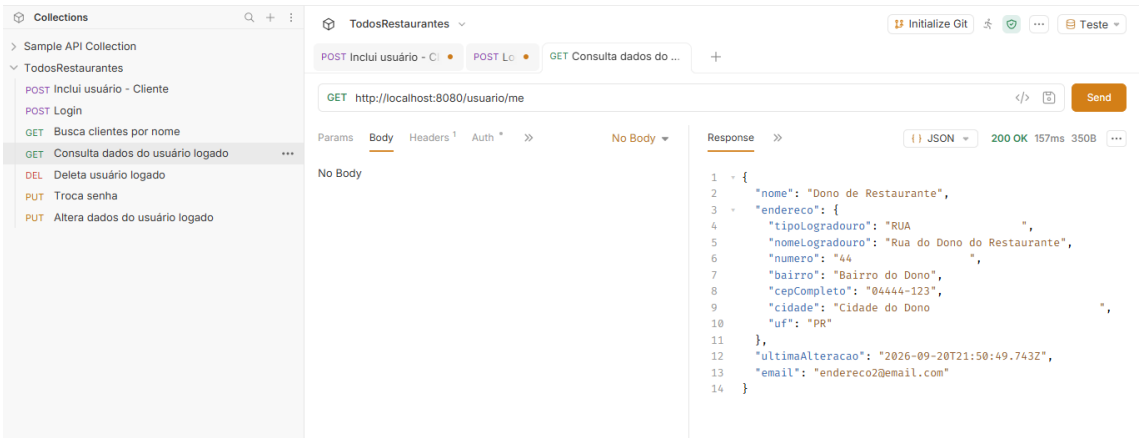
⇒ Efetuando LOGIN com usuário do tipo DONO DE RESTAURANTE:



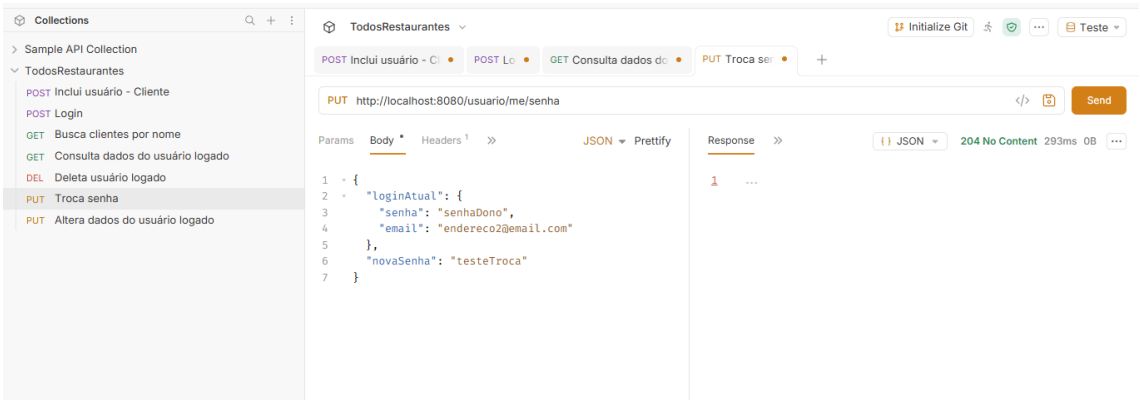
Verificando Token gerado:



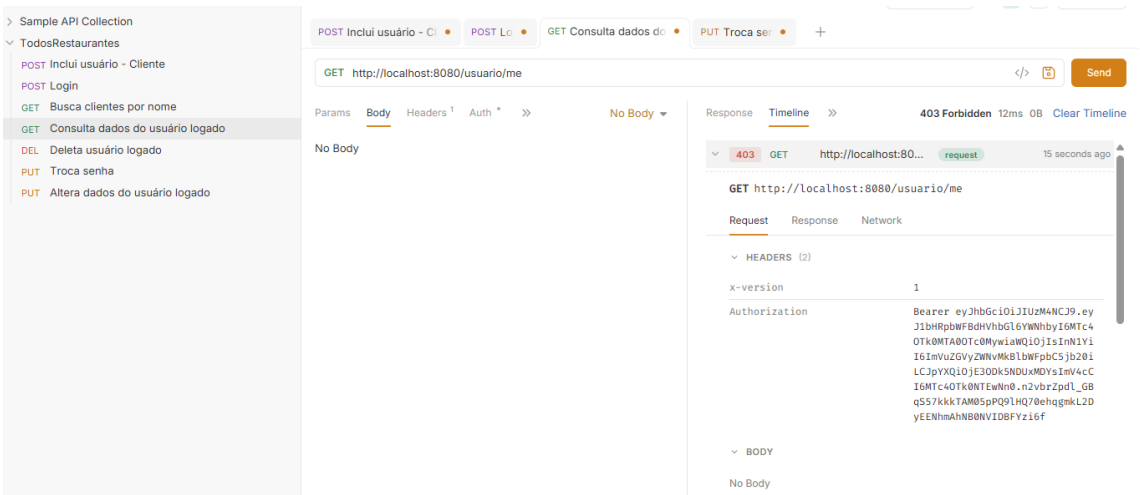
⇒ Consultando dados do usuário logado (DONO RESTAURANTE):



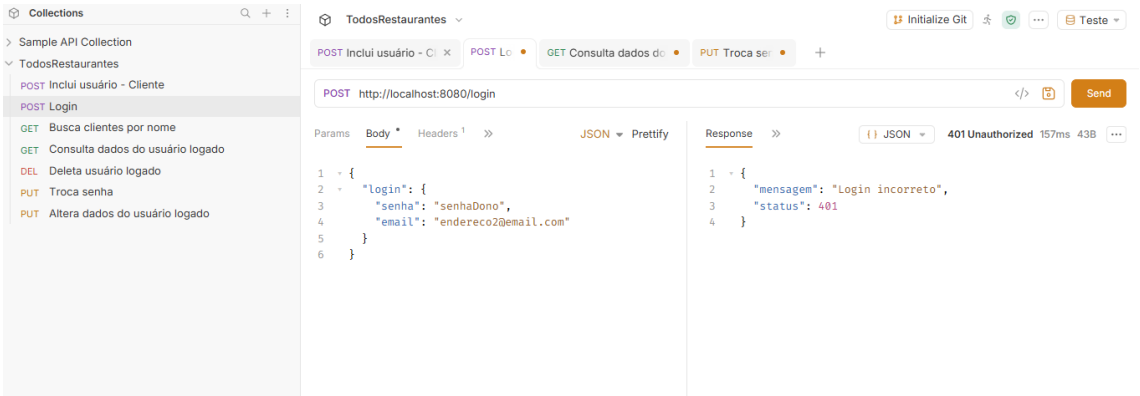
⇒ Trocando a senha do usuário logado – DONO DE RESTAURANTE – informando LOGIN atual corretamente:



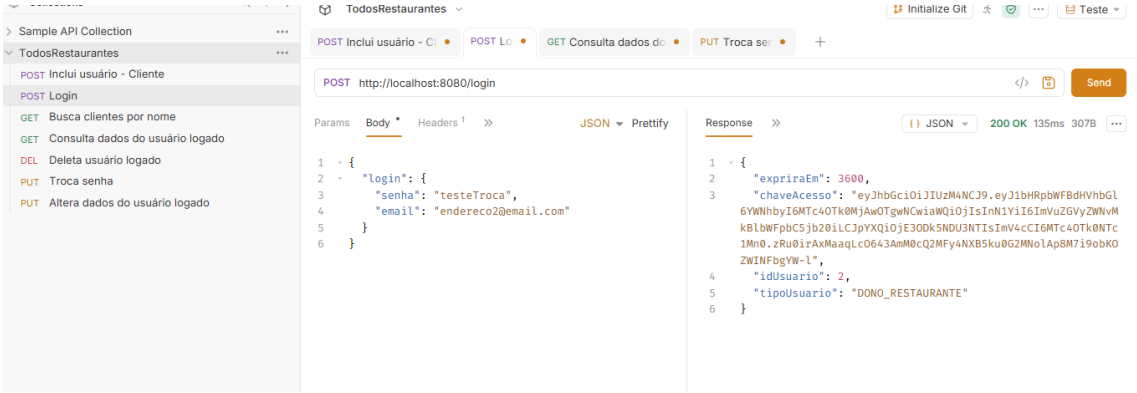
⇒ Consultando os dados do usuário logado – DONO RESTAURANTE – sem efetuar novo LOGIN:



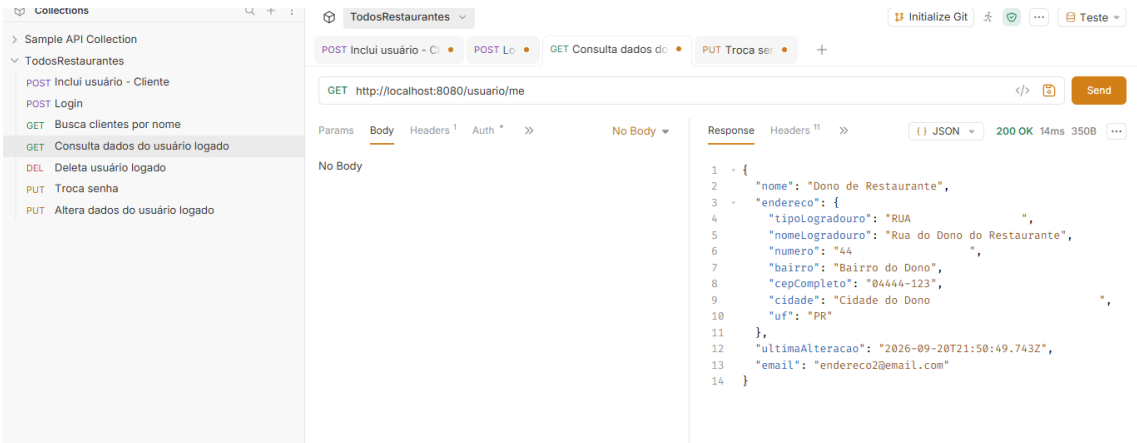
⇒ Efetuando novo LOGIN com a senha antiga:



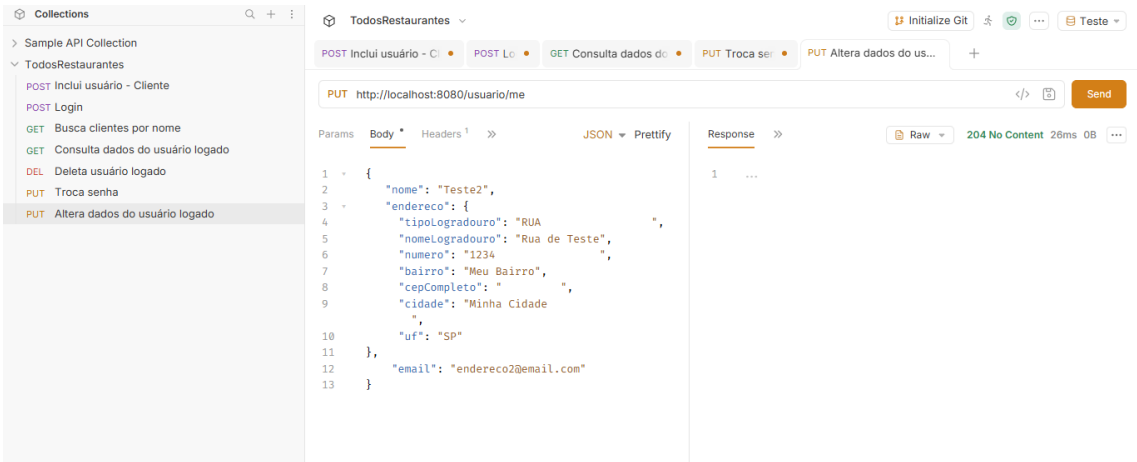
⇒ Efetuando NOVO LOGIN com a senha correta:



⇒ Consultar dados do usuário após efetuar novo LOGIN:



⇒ Alterar dados do usuário, exceto e-mail:



⇒ Consultar dados do usuário logado, após alteração:

The screenshot shows the Postman interface with a collection named 'TodosRestaurantes'. The selected request is a GET request to 'http://localhost:8080/usuario/me'. The response is a JSON object with the following structure:

```
1 {
2   "nome": "Teste2",
3   "endereco": {
4     "tipoLogradouro": "RUA",
5     "nomeLogradouro": "Rua do Dono do Restaurante",
6     "numero": "44",
7     "bairro": "Bairro do Dono",
8     "cepCompleto": "04444-123",
9     "cidade": "Cidade do Dono",
10    "uf": "PR"
11  },
12  "ultimaAlteracao": "2026-09-20T22:10:36.548Z",
13  "email": "endereço2@email.com"
14 }
```

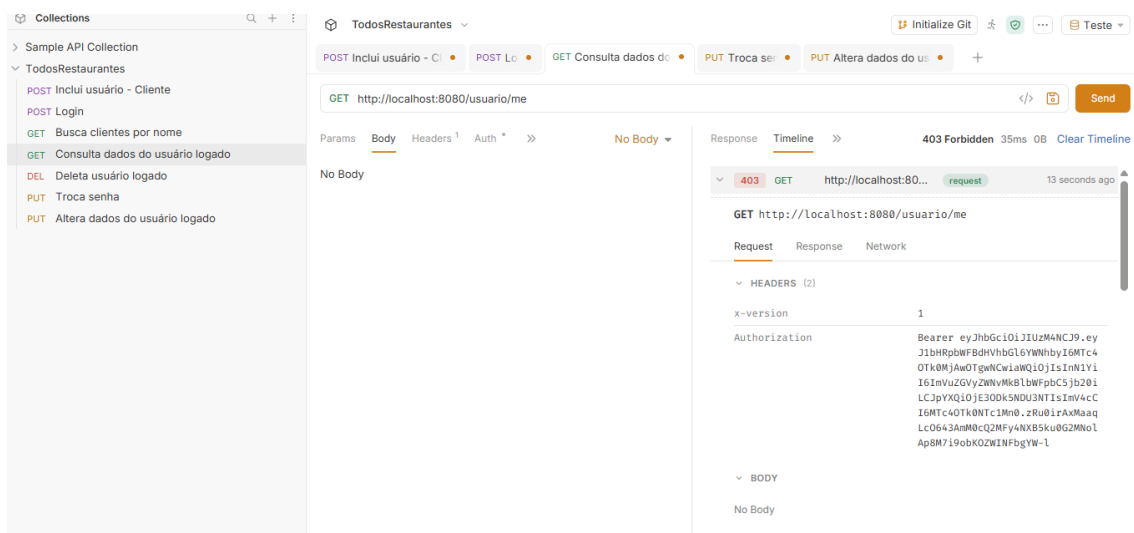
⇒ Alterando e-mail do usuário logado:

The screenshot shows the Postman interface with a collection named 'TodosRestaurantes'. The selected request is a PUT request to 'http://localhost:8080/usuario/me'. The request body is a JSON object with the following structure:

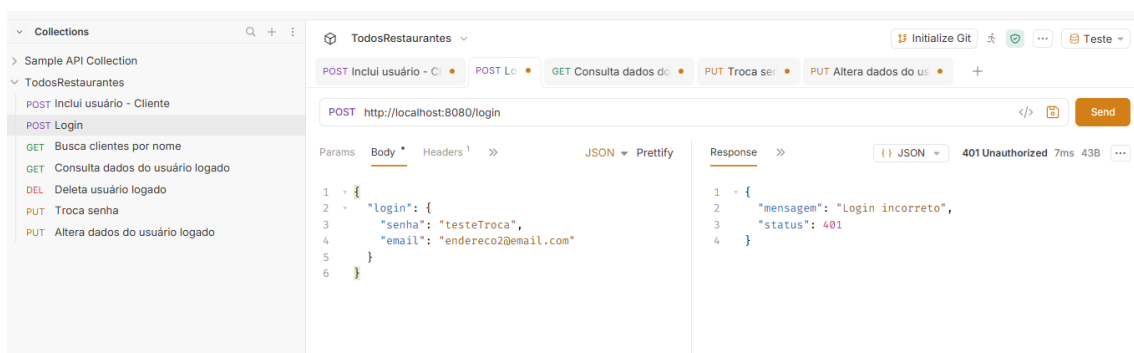
```
1 {
2   "nome": "Teste2",
3   "endereco": {
4     "tipoLogradouro": "RUA",
5     "nomeLogradouro": "Rua de Teste",
6     "numero": "1234",
7     "bairro": "Meu Bairro",
8     "cepCompleto": " ",
9     "cidade": "Minha Cidade",
10    "uf": "SP"
11  },
12   "email": "endereçoX@email.com"
13 }
```

The response status is '204 No Content'.

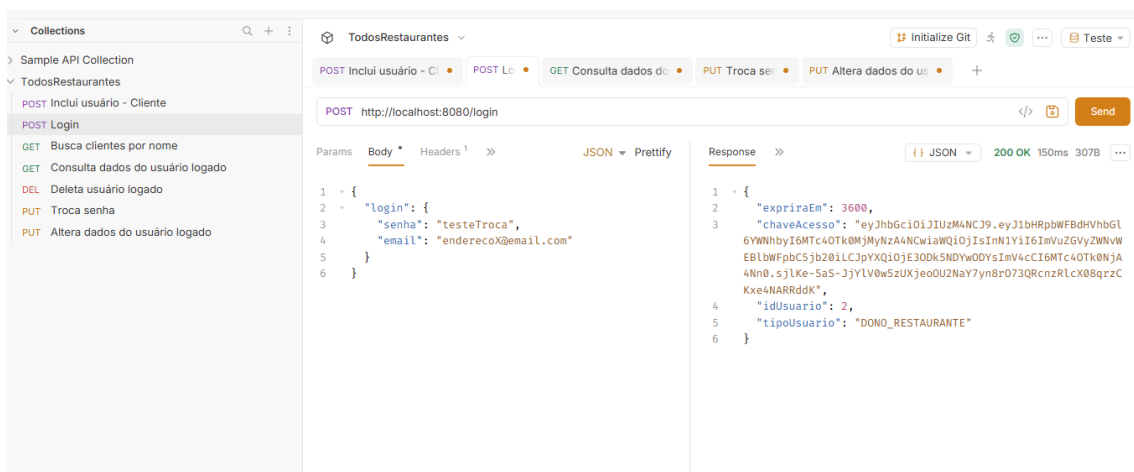
⇒ Consultando dados do usuário logado, após troca do e-mail e sem efetuar novo LOGIN:



⇒ Efetuando novo LOGIN informando o e-mail anterior:



⇒ Efetuando LOGIN efetuando o e-mail novo:



⇒ Pesquisando Clientes por NOME – usuário LOGADO DONO DE RESTAURANTE e argumento que não encontra correspondência:

The screenshot shows the Postman interface. On the left, the 'Collections' panel is open, showing a collection named 'TodosRestaurantes'. The 'GET Busca clientes por nome' endpoint is selected. The main panel shows the request details for the endpoint 'http://localhost:8080/usuario?nome=Teste'. The 'Params' tab is active, showing a single parameter 'nome' with the value 'Teste'. The 'Response' tab shows a 200 OK status with a JSON body:

```
{
  "usuarios": []
}
```

⇒ Pesquisando Clientes por NOME – usuário LOGADO DONO DE RESTAURANTE e argumento que encontra correspondência:

The screenshot shows the Postman interface. On the left, the 'Collections' panel is open, showing a collection named 'TodosRestaurantes'. The 'GET Busca clientes por nome' endpoint is selected. The main panel shows the request details for the endpoint 'http://localhost:8080/usuario?nome=Cliente'. The 'Params' tab is active, showing a single parameter 'nome' with the value 'Cliente'. The 'Response' tab shows a 200 OK status with a JSON body:

```
{
  "usuarios": [
    {
      "nome": "Cliente do Restaurante",
      "endereco": {
        "tipoLogradouro": "RUA",
        "nomeLogradouro": "Rua do Cliente",
        "numero": "123",
        "bairro": "Bairro do Cliente",
        "cepCompleto": "01234-123",
        "cidade": "Cidade do Cliente"
      },
      "uf": "SP",
      "ultimaAlteracao": "2026-09-20T21:48:57.656Z",
      "email": "endereco1@email.com"
    }
  ]
}
```

⇒ Cadastrando um novo Cliente:

The screenshot shows the Postman interface with a collection named 'TodosRestaurantes'. The selected request is 'POST Incluir usuário - Cliente' with the URL 'http://localhost:8080/usuario'. The request body is a JSON object representing a new client. The response is a 201 Created status with a 173ms response time.

```
POST http://localhost:8080/usuario
```

Body (JSON):

```
{  "nome": "Outro Cliente",  "endereco": {    "tipoLogradouro": "AVENIDA",    "nomeLogradouro": "Avenida das Desventuras",    "numero": "99",    "bairro": "Bairro do Outro",    "cepCompleto": "99999-123",    "cidade": "Cidade Alegre",    "uf": "SP"  },  "tipoUsuario": "CLIENTE_RESTAURANTE",  "login": {    "senha": "senhaAlegre",    "email": "endereço@email.com"  }}
```

Response (JSON):

```
{  "status": 201,  "message": "Created",  "data": {}}
```

⇒ Pesquisando clientes por nome – usuário logado DONO DE RESTAURANTE:

The screenshot shows the Postman interface with the same collection. The selected request is 'GET Busca clientes por nome' with the URL 'http://localhost:8080/usuario?nome=Cli'. The response is a 200 OK status with a 21ms response time and 701B of data.

```
GET http://localhost:8080/usuario?nome=Cli
```

Response (JSON):

```
{  "usuarios": [    {      "nome": "Cliente do Restaurante",      "endereco": {        "tipoLogradouro": "RUA",        "nomeLogradouro": "Rua do Cliente",        "numero": "123",        "bairro": "Bairro do Cliente",        "cepCompleto": "01234-123",        "cidade": "Cidade do Cliente",        "uf": "SP"      },      "ultimaAlteracao": "2026-09-20T21:48:57.656Z",      "email": "endereço1@email.com"    },    {      "nome": "Outro Cliente",      "endereco": {        "tipoLogradouro": "AVENIDA",        "nomeLogradouro": "Avenida das Desventuras",        "numero": "99"      }    }  ]}
```


⇒ Pesquisando usuários por nome – usuário LOGADO é CLIENTE:

The first screenshot shows a GET request to `http://localhost:8080/usuario?nome=Cli` returning a 400 Bad Request status. The response body is:

```
{
  "mensagem": "Função não permitida para o usuário",
  "status": 400
}
```

The second screenshot shows a GET request to `http://localhost:8080/usuario/me` returning a 200 OK status. The response body is:

```
{
  "nome": "Cliente do Restaurante",
  "endereco": {
    "tipoLogradouro": "RUA",
    "nomeLogradouro": "Rua do Cliente",
    "numero": "123",
    "bairro": "Bairro do Cliente",
    "cepCompleto": "01234-123",
    "cidade": "Cidade do Cliente",
    "uf": "Sp"
  },
  "ultimaAlteracao": "2026-09-20T21:48:57.656Z",
  "email": "endereco1@email.com"
}
```

⇒ Consultando dados do cliente logado:

The screenshot shows a GET request to `http://localhost:8080/usuario/me` returning a 200 OK status. The response body is:

```
{
  "nome": "Cliente do Restaurante",
  "endereco": {
    "tipoLogradouro": "RUA",
    "nomeLogradouro": "Rua do Cliente",
    "numero": "123",
    "bairro": "Bairro do Cliente",
    "cepCompleto": "01234-123",
    "cidade": "Cidade do Cliente",
    "uf": "Sp"
  },
  "ultimaAlteracao": "2026-09-20T21:48:57.656Z",
  "email": "endereco1@email.com"
}
```

⇒ Deleta usuário logado:

The first screenshot shows a DELETE request in Postman. The URL is `http://localhost:8080/usuario/me`. The Headers tab is selected, showing `x-version: 1`. The response is `204 No Content`.

The second screenshot shows the same DELETE request with the Response tab selected. The response is `204 No Content`. The Headers tab is also visible, showing `x-version: 1` and `Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpzZXQ1IiwiaWF0IjE5OTk0MjY0In0=`.

⇒ Efetuando LOGIN com usuário DONO DE RESTAURANTE:

The screenshot shows a POST request in Postman. The URL is `http://localhost:8080/login`. The Body tab is selected, showing the JSON body:

```
1 {
2   "login": {
3     "senha": "testeTroca",
4     "email": "enderecoX@email.com"
5   }
6 }
```

The response is `200 OK` with a JSON body:

```
1 {
2   "expiraEm": 3600,
3   "chaveAcesso": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpzZXQ1IiwiaWF0IjE5OTk0MjY0In0=",
4   "idUsuario": 2,
5   "tipoUsuario": "DONO_RESTAURANTE"
6 }
```

⇒ Pesquisando usuários por nome – usuário logado DONO DE RESTAURANTE:

The screenshot shows the Postman interface with a collection named 'TodosRestaurantes'. The selected endpoint is 'GET http://localhost:8080/usuario?nome=Cli'. The request is successful, returning a 200 OK status. The response body is a JSON object containing a list of users, with the first user being the one whose name was searched for.

Name	Value	Description
nome	Cli	

```
1 {
2   "usuarios": [
3     {
4       "nome": "Outro Cliente",
5       "endereco": {
6         "tipoLogradouro": "AVENIDA",
7         "nomeLogradouro": "Avenida das Desventuras",
8         "numero": "99",
9         "bairro": "Bairro do Outro",
10        "cepCompleto": "99999-123",
11        "cidade": "Cidade Alegre",
12        "uf": "SP"
13      },
14      "ultimaAlteracao": "2026-09-20T22:20:33.187Z",
15      "email": "endereço@email.com"
16    }
17  ]
18 }
```

⇒ Omitindo o Login atual na requisição de troca de senha:

The screenshot shows the Postman interface with a collection named 'TodosRestaurantes'. The selected endpoint is 'PUT http://localhost:8080/usuario/me/senha'. The request body contains only the new password: `{ "novaSenha": "testeTroca" }`. The request fails with a 400 Bad Request status. The response body is a JSON object with a message and status.

```
1 {
2   "mensagem": "Obrigatório informar os dados do Login atual",
3   "status": 400
4 }
```

⇒ Omitindo a nova senha na requisição de troca de senha:

The screenshot shows the Postman interface with a collection named 'TodosRestaurantes'. The selected endpoint is 'PUT http://localhost:8080/usuario/me/senha'. The request body contains the current login and email, but omits the new password: `{ "loginAtual": { "senha": "senhaDono", "email": "endereço2@email.com" } }`. The request fails with a 400 Bad Request status. The response body is a JSON object with a message and status.

```
1 {
2   "mensagem": "Obrigatório informar os dados do Login",
3   "status": 400
4 }
```

⇒ Acionando os endpoints informando versão incorreta:

The screenshot shows the Postman interface with the 'TodosRestaurantes' collection selected. The active endpoint is 'POST http://localhost:8080/usuario'. The 'Headers' tab is active, showing 'x-version' with a value of '2'. The 'Response' tab shows a 500 Internal Server Error with the following JSON body:

```
1 {
2   "mensagem": "Ocorreu um erro interno no servidor. Tente novamente mais tarde.",
3   "status": 500
4 }
```

The screenshot shows the Postman interface with the 'TodosRestaurantes' collection selected. The active endpoint is 'GET http://localhost:8080/usuario?nome=Cliente'. The 'Headers' tab is active, showing 'x-version' with a value of '2'. The 'Response' tab shows a 500 Internal Server Error with the following JSON body:

```
1 {
2   "mensagem": "Ocorreu um erro interno no servidor. Tente novamente mais tarde.",
3   "status": 500
4 }
```

The screenshot shows the Postman interface with the 'TodosRestaurantes' collection selected. The active endpoint is 'POST http://localhost:8080/login'. The 'Headers' tab is active, showing 'x-version' with a value of '2'. The 'Response' tab shows a 500 Internal Server Error with the following JSON body:

```
1 {
2   "mensagem": "Ocorreu um erro interno no servidor. Tente novamente mais tarde.",
3   "status": 500
4 }
```

The screenshot shows the Postman interface with the 'TodosRestaurantes' collection selected. The active endpoint is 'GET http://localhost:8080/usuario/me'. The 'Headers' tab is active, showing 'x-version' with a value of '2'. The 'Response' tab shows a 500 Internal Server Error with the following JSON body:

```
1 {
2   "mensagem": "Ocorreu um erro interno no servidor. Tente novamente mais tarde.",
3   "status": 500
4 }
```

Interface do Postman mostrando a coleção "TodosRestaurantes". A aba selecionada é "DELETE Deleta usuário" com o endpoint `http://localhost:8080/usuario/me`. O método é DELETE. A aba "Headers" está selecionada, mostrando o cabeçalho `x-version` com o valor `2`. A aba "Response" mostra o status `500 Internal Server Error` com o tempo de resposta de `12ms` e o tamanho de `92B`. O corpo da resposta é:

```
1 {
2   "mensagem": "Ocorreu um erro interno no servidor. Tente novamente mais tarde.",
3   "status": 500
4 }
```

Interface do Postman mostrando a coleção "TodosRestaurantes". A aba selecionada é "PUT Alterar dados do usuário" com o endpoint `http://localhost:8080/usuario/me/senha`. O método é PUT. A aba "Headers" está selecionada, mostrando o cabeçalho `x-version` com o valor `2`. A aba "Response" mostra o status `500 Internal Server Error` com o tempo de resposta de `13ms` e o tamanho de `92B`. O corpo da resposta é:

```
1 {
2   "mensagem": "Ocorreu um erro interno no servidor. Tente novamente mais tarde.",
3   "status": 500
4 }
```

Interface do Postman mostrando a coleção "TodosRestaurantes". A aba selecionada é "PUT Alterar dados do usuário" com o endpoint `http://localhost:8080/usuario/me`. O método é PUT. A aba "Headers" está selecionada, mostrando o cabeçalho `x-version` com o valor `2`. A aba "Response" mostra o status `500 Internal Server Error` com o tempo de resposta de `28ms` e o tamanho de `92B`. O corpo da resposta é:

```
1 {
2   "mensagem": "Ocorreu um erro interno no servidor. Tente novamente mais tarde.",
3   "status": 500
4 }
```

⇒ Incluindo usuário com omissão de dado obrigatório:

Postman interface showing a failed POST request to `http://localhost:8080/usuario`. The request body is a JSON object with a `login` field, but it is missing the `senha` field. The response is a 400 Bad Request with a message: "Obrigatório informar o nome do cliente".

```
POST http://localhost:8080/usuario
```

Body (JSON):

```
{  "endereco": {    "tipoLogradouro": "AVENIDA",    "nomeLogradouro": "Avenida das Desventuras",    "numero": "99",    "bairro": "Bairro do Outro",    "cepCompleto": "99999-123",    "cidade": "Cidade Alegre",    "uf": "SP"  },  "tipoUsuario": "CLIENTE_RESTAURANTE",  "login": {    "senha": "senhaAlegre",    "email": "endereco@email.com"  }}
```

Response (JSON):

```
{  "mensagem": "Obrigatório informar o nome do cliente",  "status": 400}
```

Postman interface showing a failed POST request to `http://localhost:8080/usuario`. The request body is a JSON object with a `nome` field, but it is missing the `senha` field. The response is a 400 Bad Request with a message: "Obrigatório informar usuário e senha".

```
POST http://localhost:8080/usuario
```

Body (JSON):

```
{  "nome": "Outro Cliente",  "endereco": {    "tipoLogradouro": "AVENIDA",    "nomeLogradouro": "Avenida das Desventuras",    "numero": "99",    "bairro": "Bairro do Outro",    "cepCompleto": "99999-123",    "cidade": "Cidade Alegre",    "uf": "SP"  },  "tipoUsuario": "CLIENTE_RESTAURANTE"}
```

Response (JSON):

```
{  "mensagem": "Obrigatório informar usuário e senha",  "status": 400}
```

6. Instruções para executar a aplicação com Docker Compose

Na raiz do projeto, criar o arquivo '.env' e colocar a variável de ambiente JWT_SECRET com a chave para geração do token JWT. Por exemplo:

```
JWT_SECRET=MinhaChaveUltraSecretaEComMaisDe64CaracteresAleatorios123456
```